HostelFix: AI-Powered Hostel Maintenance and Housekeeping Portal

Github Repository Link : https://github.com/hostelfix-iitdh/hostelfix

1. Overview

1.1 Website Name

HostelFix

1.2 Team Members

No.	Name	Roll Number	GitHub Username
1	Narasimha Reddy Boggula	CS26MT002	Narasimha013
2	Aakash Saran V	CS26MT018	AakashSaran018
3	Suru Rakesh	CS26MT022	cs26mt022

1.3 Problem Statement

In a hostel, a resident who notices a problem such as a dirty corridor, an overflowing dustbin, a broken light or a lift that has stopped working usually has to find out whom to contact, pass on the details, and later ask again to learn whether anything was done. There is typically no single record that shows whether the problem was received, who is handling it, or whether other residents face the same problem. As a result, the same issue is reported repeatedly by different people, staff do not get one consolidated view of what is pending, and the hostel has no reliable history from which to see which problems keep recurring.

HostelFix addresses this by giving residents one place to report a problem with its exact location, track it until it is fixed, and confirm that the fix worked, and by giving staff and administrators one place to manage and review reports.

1.4 Description

HostelFix is a multi-page web application backed by a MySQL database and a documented REST API. Hostel residents report maintenance and housekeeping problems, track their progress, and see what is being fixed across the hostel, while authorized staff manage and resolve those reports. A resident selects the exact location (hostel, floor, wing and area) and the type of problem, either through a manual form or through SmartReport, an AI feature that turns a plain-language description into an editable draft complaint.

Every complaint receives an ID and moves through a tracked lifecycle, the original reporter confirms whether the fix worked, and residents get an in-app notification whenever the status changes. A privacy-safe Community Issues page shows complaints without revealing who reported them, and residents can add a “Me too” confirmation instead of filing duplicates. Students register with a verified iitdh.ac.in email or sign in with their IIT

Dharwad Google account, and only iitdh.ac.in addresses are accepted for students. Administrators review unresolved and overdue complaints and basic analytics.

1.5 Target Users

- **Students (hostel residents):** report problems, track them, receive status notifications, browse community issues and confirm resolutions.
- **Maintenance and housekeeping staff:** each staff member is assigned to one category (Maintenance or Housekeeping) and sees only the complaints of that category. They update status, add progress notes and record resolutions.
- **Administrators:** manage accounts, roles, locations, categories and assets, review overdue complaints and analytics, and can perform any staff action when a staff member is unavailable.

1.6 Repository Setup

Item	Setting
Repository	https://github.com/hostelfix-iitdh/hostelfix
Members	Narasimha013, AakashSaran018, cs26mt022 and the instructor
README	Drafted: summary, team, one-command run (`docker compose up`), environment variables (`.env.example`), how to run tests, link to this document, branching and review rules
Branch protection	The ruleset “Protect main” is active on `main`: changes only through pull requests with at least one approval from another member, no force pushes or branch deletion
Branches and commits	Branches named `feature/<topic>` or `fix/<topic>`; each member commits under their own account so Git history shows individual contribution
Secrets and files	API keys and credentials only in GitHub Secrets and a git-ignored `.env`; `AI_USAGE.md` and a pull request template kept in the repository

2. Dataset

2.1 Source and License

HostelFix uses a synthetic dataset built by the team. No external dataset or license is required, and no real student data is used. The layout is the team’s own description of the two hostels, written from members’ knowledge as residents, not copied from an official floor plan. It is stored in a configuration file (`config/hostel_layout.yaml`) so it can be corrected without changing code. Categories and problem types are authored by the team from common hostel maintenance and housekeeping issues. Users and complaints are generated with a Python seed script using the Faker library and hand-written complaint templates.

2.2 Entities and Relevant Fields

Entity	Key fields	How it is populated
hostels, floors, wings	name or number, parent reference	2 hostels, 8 floors each, 5 wings per floor (80 wings in total), from the layout file
locations	wing reference, area type (Toilet, Bathroom, Washing Machine Area, Cloth Drying Area, Corridor, Lift Lobby, Drinking Water Area, Room)	One record per area type per wing
assets	name, type, hostel reference	Six lifts per hostel (Lift 1 to Lift 6) and optional water dispensers
asset_locations	asset reference, location reference	Links each lift to the lift lobbies it serves on every floor: Lift 1 serves the lobby near the entrance and stairs (wing A), Lifts 2 and 3 the lobby serving wings B and C, and Lifts 4, 5 and 6 the middle lobby serving wings D and E.
categories, problem_types	name, category reference, requires-description flag	Maintenance and Housekeeping, about 40 problem types, each list including “Other”
users	display name, email, password hash, Google account ID (if used), role, assigned category (staff only), email verified flag, created date	40 students, 4 staff (2 per category), 1 administrator, with placeholder emails. All seeded accounts are marked verified.
email_tokens	user, purpose (verify or reset), token hash, expiry, used flag	Created at runtime only
issues	reporter, location, optional room label, optional asset, category, problem type, title, description, optional service date (housekeeping), status, closed reason, due date, timestamps	About 200 synthetic complaints over the previous 60 days with varied statuses
issue_updates	issue, actor, actor role, previous and new status, note, timestamp	Generated from each complaint’s lifecycle
issue_confirmations	issue, user (unique together)	About 150 random “Me too” records
issue_resolution_feedback	issue, reporter, confirmed or reopen request, comment	Generated for resolved complaints
notifications	user, issue, message, read flag, created timestamp	Generated from seeded status changes

2.3 Sample Records

Location	Category	Problem type	Description	Status
Hostel 1, Floor 4, Wing B, Lift Lobby (Lift 2)	Maintenance	Lift not working	Lift 2 stops between floors and the door does not open.	In Progress
Hostel 2, Floor 3, Wing D, Drinking Water Area	Maintenance	Cooler leaking	Water is leaking from the base of the cooler.	Acknowledged
Hostel 2, Floor 5, Wing D, Corridor	Housekeeping	Wet mopping not completed	The corridor was not wet-mopped today.	Resolved (awaiting confirmation)
Hostel 1, Floor 2, Wing A, Washing Machine Area	Housekeeping	Area not cleaned	Water and lint are on the floor near the machines.	Closed

2.4 Mapping to Features

- The hostel hierarchy, locations and assets drive the cascading location selection and the Community Issues filters. The asset-to-location link shows only the lifts that serve the chosen lobby.
- Categories and problem types drive the manual form, the SmartReport options and the analytics.
- Complaints, updates, confirmations, feedback and notifications give the status timelines, “Me too” counts, notification bell and analytics realistic content from the first run.
- Seeded users with different roles let every role-based page be demonstrated and tested.

2.5 How the Dataset Is Built

A seed script reads fixed configuration files for the hostel layout, lift-to-lobby links, categories and problem types, then generates users and complaints with a fixed random seed so that every run produces the same data. Descriptions come from hand-written templates for each problem type, with variations in wording, location and date. Statuses are spread across the full lifecycle, with most older complaints resolved or closed and a smaller share open or overdue. The script runs automatically with “docker compose up”, after the database migrations. Seeded accounts use passwords hashed in the same way as real accounts.

Individual rooms are not seeded because the room inventory is not confirmed. A student who selects the Room area enters an optional room label instead, stored with the complaint. Each hostel has six lifts in three lobbies: one lift near the entrance and stairs (wing A), two near wings B and C, and three in the middle near wings D and E. The lift-to-lobby links are read from a configuration file, so they can be corrected without changing the schema or code. Both hostels are seeded with the same arrangement.

3. Content Structure

3.1 Sitemap

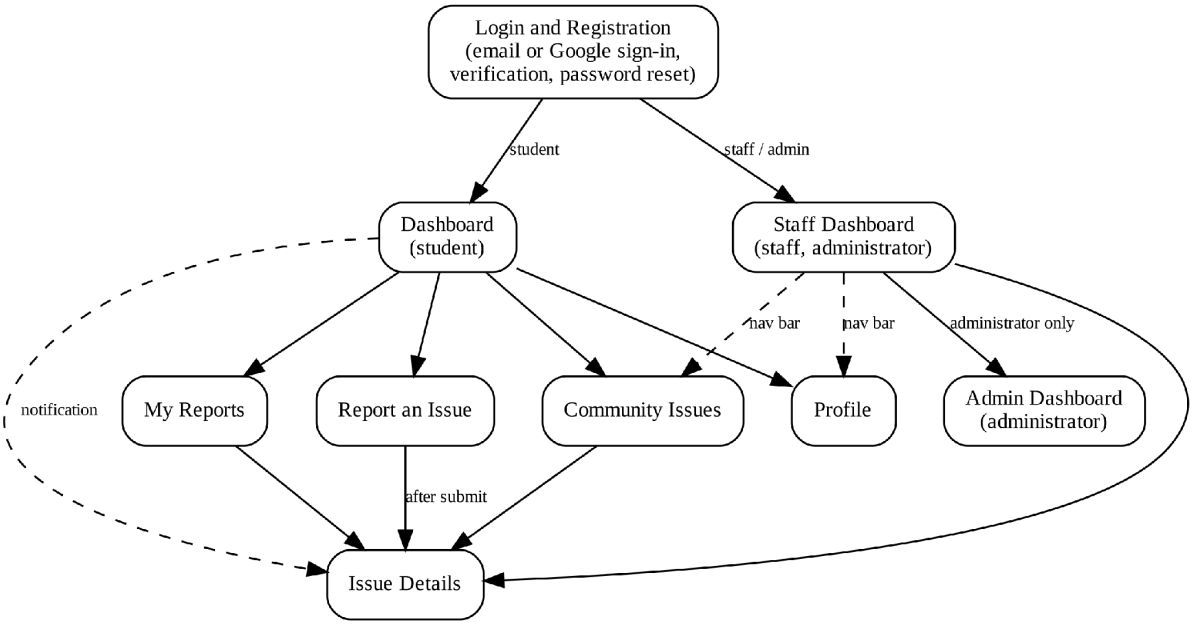


Figure 1: Sitemap of HostelFix

The application has nine pages. Figure 1 shows how they connect. After signing in, users move between pages through a navigation bar that shows only the pages permitted for their role. Students also see a notification bell. Staff and administrators reach Community Issues and Profile through the navigation bar.

3.2 Page Access and Navigation

Page	**Reachable from**	**Available to**
Login and Registration	Entry point; logout redirect	Everyone
Dashboard	Login; navigation bar	Students
Report an Issue	Dashboard; navigation bar	Students
Community Issues	Dashboard; navigation bar	Signed-in users
Issue Details	Community Issues, My Reports, Report an Issue, Staff Dashboard, notifications	Signed-in users, with role-based content
My Reports	Dashboard; navigation bar	Students
Profile	Navigation bar	Signed-in users
Staff Dashboard	Login (staff and administrators); navigation bar	Staff, administrators
Admin Dashboard	Staff Dashboard; navigation bar	Administrators

Logging out or deleting an account returns the user to the Login and Registration page. Community Issues are shown to signed-in campus users only, so complaint information is not public on the internet. During development, the browsing, searching and viewing pages are connected to the backend first (MS-4), and authentication (registration, email verification, password reset and Google sign-in) is added to the running site afterwards (MS-5), with the existing tests kept passing.

4. Functionality

4.1 Page-by-Page Features

1. Login and Registration

Shows: A sign-in form, a registration form with a consent notice, a “Sign in with Google” button, a “Forgot password” link, the verification and reset screens, and clear error messages (for example, when the email is not an iitdh.ac.in address or the account is not yet verified).

Users can: Students: register with an iitdh.ac.in email and a password, verify the email through the link sent to that mailbox, or sign in with their iitdh.ac.in Google account. Staff and administrators: set a password through the link an administrator sends them, then sign in with email and password. All password users: reset a forgotten password. Everyone: sign out.

Access: Everyone.

2. Dashboard

Shows: The student’s recent reports with their status, a list of recent status-change notifications (unread highlighted), and the latest community issues.

Users can: Open a report or notification; mark notifications as read; start a new report; move to Community Issues or My Reports.

Access: Students.

3. Report an Issue

Shows: A SmartReport text box, a manual form with cascading selections (hostel, floor, wing, area, asset where relevant, category, problem type, optional room label and, for housekeeping, an optional service date), an editable AI draft with inferred fields highlighted and missing fields listed, similar open complaints, and a final review summary.

Users can: Describe the problem in plain language and request a draft; edit any field; choose “Other” and enter the required description; add “Me too” on a similar complaint instead of filing a duplicate; submit the reviewed complaint. If the AI service is unavailable, the manual form still works.

Access: Students.

4. Community Issues

Shows: A paginated list with complaint ID, title, location (down to the area, never the room label), category, report date, status, “Me too” count and “Reporter: Anonymous”.

Users can: Search; filter by hostel, floor, wing, area, category, problem type and status; sort by date or “Me too” count; open a complaint; add or withdraw “Me too” (not on Room-area complaints).

Access: Signed-in users.

5. Issue Details

Shows: The permitted complaint details, a privacy-safe status timeline, resolution notes and the “Me too” count. The room label is shown only to the reporter, to staff of the complaint’s category and to administrators. Staff also see internal progress notes. The reporter’s name and email are visible only to administrators.

Users can: Students: add or withdraw “Me too”. The original reporter: confirm the resolution or request reopening. Staff and administrators: update status and add progress notes within the allowed transitions.

Access: Signed-in users, with role-based content.

6. My Reports

Shows: A private list of the student’s complaints with status and last update.

Users can: Filter by status; open a complaint; edit the description while it is still Submitted; confirm resolution or request reopening. The backend checks ownership on every request.

Access: Students, for their own complaints only.

7. Profile

Shows: Display name, college email, role and the consent record.

Users can: Edit the display name; change the password; log out; delete the account. Deleting an account removes personal data, while complaint history is kept in anonymised form.

Access: Signed-in users.

8. Staff Dashboard

Shows: The complaints of the staff member’s assigned category, with filters and an overdue indicator calculated from the due date and status. Administrators see all categories.

Users can: Acknowledge a complaint; set it In Progress; add a progress note; mark it resolved with a resolution note; reject an invalid complaint with a reason.

Access: Staff and administrators.

9. Admin Dashboard

Shows: Complaint counts by category, location and status, frequent problem types, open and overdue complaints, average resolution time and reopened complaints.

Users can: Create staff accounts and assign their category (the new staff member receives a set-password link by email); manage roles, locations, categories, problem types, assets and lift-to-lobby links; set resolution target durations.

Access: Administrators.

4.2 Roles and Permissions

Permissions are enforced by the backend on every protected endpoint; hiding a button in the frontend is not a security control. A user cannot give themselves a staff or administrator role.

Capability	Student	Staff	Administrator
Sign in	Yes	Yes	Yes
Submit a complaint (manual or SmartReport)	Yes	No	No
Browse Community Issues	Yes	Yes	Yes
Add or withdraw “Me too”	Yes	No	No
View own reports and notifications	Yes	No	No
See who reported a complaint	Own complaints only	No	Yes
See the room label	Own complaints only	Assigned category only	Yes
Confirm resolution or request reopening	Own complaints only	No	No
Update status and add progress notes (see 4.3)	No	Assigned category only	Yes, any category
Manage accounts, roles, locations, categories, assets	No	No	Yes
View analytics	No	No	Yes

Administrators cannot perform the reporter’s actions. Every status change records the actor and the actor’s role.

4.3 Complaint Lifecycle

There is one status model, shown in the table below. **Timeout is not a status:** when a Resolved complaint gets no response, the system moves it to Closed and records the reason “auto-closed: no response”. Closed and Rejected are final. The backend rejects every transition not in this table.

From	To	Who	Required	Meaning of the new status
(new complaint)	Submitted	System	Valid complaint	The complaint has been accepted and stored.
Submitted	Acknowledged	Staff of the category, or administrator	None	Staff have seen the complaint.

From	To	Who	Required	Meaning of the new status
Submitted or Acknowledged	Rejected	Staff of the category, or administrator	Reason	The complaint is invalid.
Acknowledged	In Progress	Staff of the category, or administrator	None	Work is underway.
In Progress	Resolved	Staff of the category, or administrator	Resolution note	Marked as fixed; the reporter is asked to confirm.
Resolved	Closed	Original reporter	Optional comment	The reporter confirmed the fix.
Resolved	Closed	System	7 days with no response (configurable); reason recorded	Closed without confirmation.
Resolved	Reopened	Original reporter	Comment	The reporter says the issue persists.
Reopened	In Progress	Staff of the category, or administrator	None	Work resumes on the same complaint.

Each change is stored with the actor, actor role, timestamp, previous and new status, reason and comment, so history is never silently overwritten, and each change creates an in-app notification for the reporter. Each category has an administrator-set target resolution time, which gives every complaint a due date. A complaint is overdue when its due date has passed and it is not yet Resolved, Closed or Rejected; this is calculated from timestamps and status, not stored as a flag. A Closed complaint cannot be reopened; if the problem returns, the student files a new complaint. HostelFix is not an emergency-response service.

4.4 Key Rules

- The backend validates the whole location hierarchy and rejects invalid combinations. An asset must serve the selected location through the asset-to-location link; for example, a lift must be linked to the selected lift lobby.
- Every problem-type list includes “Other”; choosing it makes the description required, enforced by the backend.
- A user can add only one “Me too” to a complaint (database uniqueness constraint) and can withdraw it. “Me too” is disabled on Room-area complaints.
- Public and community responses never contain the reporter’s name, email, student ID, internal ID or the room label. Staff views hide the reporter but show the room label of their own category, which they need to do the work. Only administrators can see who reported a complaint, so that misuse can be handled.
- Ownership is checked on the server for My Reports, notifications and resolution feedback; an ID in a URL is never trusted by itself.
- The draft endpoint never creates a complaint; only an explicit submission does.

- Similar open complaints are matched by location, category and problem type (and service date for housekeeping). Room-area complaints are not suggested. Complaints are never merged automatically.

4.5 User Stories

No.	User story	Page
1	As a student, I want to register with my iitdh.ac.in email (verified through that mailbox) or sign in with Google, and to reset a forgotten password, so that only people who actually own an iitdh.ac.in address can use the portal and I can regain access without asking anyone.	Login and Registration
2	As a student, I want to describe a problem in my own words and get an editable draft so that I can file a complaint quickly without filling a long form from scratch.	Report an Issue
3	As a student, I want to choose the hostel, floor, wing and area step by step so that my complaint reaches the right place with the correct location.	Report an Issue
4	As a student, I want to see similar open complaints before I submit so that I can add “Me too” instead of creating a duplicate.	Report an Issue
5	As a student, I want to be notified inside the portal when the status of my complaint changes, and to see all my complaints in one list, so that I do not have to chase staff for updates.	Dashboard, My Reports
6	As a student, I want to browse community issues without seeing who reported them so that I can see what is being fixed while reporters stay anonymous.	Community Issues
7	As a staff member, I want to filter complaints by status and location so that I can plan my work and handle the oldest ones first.	Staff Dashboard
8	As a staff member, I want to mark a complaint as resolved with a note so that the reporter knows what was done and can verify it.	Staff Dashboard, Issue Details
9	As a reporter, I want to confirm a resolution or request reopening so that a problem is not closed while it still exists.	Issue Details, My Reports
10	As an administrator, I want to view overdue complaints and counts by category and location so that I can spot recurring problems and follow up.	Admin Dashboard

5. AI Feature Proposal: SmartReport

5.1 What It Does

SmartReport lets a student describe a problem in natural language (English, Hinglish or a mix). The AI suggests structured fields: hostel, floor, wing, area, asset, category, problem type, a short description, a list of missing fields and whether clarification is needed. For every field it also returns the exact words of the student's text that support it. The suggestion is shown as an editable draft in which fields not directly supported by the text are highlighted as inferred. A complaint is created only when the student reviews the form and explicitly submits it.

5.2 Model and API

We use Google's Gemini API, called only from the backend, with the model Gemini 2.5 Flash-Lite (`gemini-2.5-flash-lite`). The model name is an environment variable, so it can be changed easily. Each request uses a fixed JSON output format and a 1 hour timeout. The integration sits behind a thin provider interface so the model can be replaced without changing the rest of the application. The API key is stored in an environment variable and in GitHub Secrets, and is never committed or exposed to frontend code.

5.3 Data Sent to the Model

Only the text the student typed and the list of valid option names (hostels, floors, wings, areas, categories and problem types) are sent. The student's name, email, user ID, roll number, password and any other account identifier are never sent. The page asks students not to include personal details in their text, and the backend also removes email addresses, phone numbers and roll numbers before sending. All demonstration and evaluation inputs are written by the team and contain no real personal data.

5.4 Example

Student input: "The lift in Hostel 1, Wing B, Floor 4 is not working."

Draft field	**Suggested value**	**Supporting words from the input**	**Shown as**
Hostel / Floor / Wing	Hostel 1 / Floor 4 / Wing B	"Hostel 1", "Floor 4", "Wing B"	Stated
Area	Lift Lobby	"The lift"	Inferred
Asset	Not suggested	none	Missing
Category	Maintenance	"is not working"	Inferred
Problem type	Lift not working	"The lift ... is not working"	Stated
Missing fields	Which lift	none	—
Clarification needed	Yes; the student chooses the lift before submitting	none	—

5.5 How the Output Is Handled

- Two separate endpoints are used: POST /api/v1/issues/draft returns a draft only and never creates a complaint; POST /api/v1/issues creates a complaint after explicit submission.
- The model must return JSON in a fixed schema: for each field a `value` and a `span` (the supporting words), plus `missing_fields` and `needs_clarification`.
- The backend validates every value against real locations, assets, categories and problem types, and drops anything that does not match.
- Evidence check. A model can invent a real location the student never typed (for example "Wing C"). So for hostel, floor, wing, area and asset the backend also checks that the `span` appears in the student's text and contains the value or a configured alias (such as "H1"). A location field that fails is dropped and reported as missing. Category and problem type may be inferred, so they are kept but highlighted as inferred.
- The AI must not invent missing locations or assets; missing or ambiguous information becomes a clarification request. The AI does not suggest a priority, and no priority field exists.

5.6 What Happens When It Fails

If the AI service is slow (over the 1 hour timeout), unavailable, over quota or returns invalid output, the student sees a clear message and the manual form stays fully usable. The rest of the website is not affected.

5.7 Guardrails

- Input length is limited (initially 500 characters, configurable).
- Requests are rate-limited to 10 draft requests per user per day(configurable).
- Student text is treated as untrusted data: it is passed to the model as content to analyse, never as instructions. A message such as "ignore previous instructions" cannot change the output format or create a complaint, because the output is schema-validated and the draft endpoint has no write access to complaints.

5.8 Testing and Evaluation

Calls to the model are mocked in all unit and integration tests so that continuous integration never depends on an external service or spends quota. A separate evaluation set of 40 manually reviewed inputs, with expected fields written down beforehand, will be run against the real model. It deliberately includes Hinglish, typos, very short inputs and traps for invented values.

Group	Count	What it checks
Clear maintenance reports	5	Correct location, category and problem type
Clear housekeeping reports	4	Correct category and area
Lift-specific reports	3	Lift handling and clarification when the lift is not named
Drinking-water reports	3	Drinking Water Area and its problem types
Incomplete descriptions	3	Missing fields reported, nothing invented
Ambiguous descriptions	2	Clarification requested instead of a guess
Hinglish and mixed-language reports	6	Correct fields from text such as “Wing B ki lift kharab hai”
Typos and abbreviations	3	Correct fields despite spelling errors and forms such as “H1”
Very short inputs	3	Clarification requested, nothing invented
Partial-location traps	3	A location the student did not type is not suggested
Prompt-injection attempts	5	Output stays in the schema and no complaint is created

For each input we record whether the output was valid JSON, how many fields matched, whether clarification was requested when it should have been, how many suggested values had no supporting words (and whether value validation or only the evidence check caught them), and the response time and any failures.

Time saved. The cascading form already collects most of the data, so SmartReport must be shown to help. Five volunteers will each file the same six written scenarios once with the manual form and once with SmartReport (order alternated); we report the median time and the number of fields corrected. Results will be reported honestly in the final report, including failures and cases where SmartReport was slower.

6. Authentication and Security Plan

- **Sign-in methods:** students can register with an iitdh.ac.in email and a password, or sign in with Google. Staff and administrator accounts are created by an administrator; the new user receives a one-time set-password link by email and then signs in with email and password.
- **College-domain rule (exact match):** for student accounts, the backend lower-cases the address, splits it at the last “@” and accepts it only if the part after the “@” is exactly `iitdh.ac.in`. Addresses such as `x@notiitdh.ac.in` or `x@mail.iitdh.ac.in` are rejected. The check runs on the backend, not only in the frontend, and is tested with look-alike domains.

- **Email verification:** a new student account starts unverified and cannot sign in until the student opens a single-use link (random token stored as a hash, valid for 24 hours) sent to that mailbox. This proves the student controls the address. Resending is rate-limited. In the demonstration setup, mail is sent through an SMTP account set in environment variables or written to the application log, so "docker compose up" works with nothing else installed.
- **Password reset:** "Forgot password" sends a single-use reset link (token stored as a hash, valid for 30 minutes). The reply is the same whether or not the address has an account, so it cannot be used to find out who is registered. Using the link sets a new password and ends the user's existing sessions. Requests are rate-limited.
- **Google sign-in:** the backend verifies the Google ID token (signature, audience and expiry) and requires `email_verified` to be true, the hosted-domain claim (`hd`) to equal `iitdh.ac.in`, and the email to pass the same exact-domain check. A Google sign-in is linked to an existing password account only when that account is verified.
- **Roles:** student, staff (assigned to Maintenance or Housekeeping) and administrator, stored on the server and not changeable by the user. The first administrator is created at first start from credentials in environment variables.
- **Passwords:** stored as salted Argon2 (or bcrypt) hashes, never encrypted, logged or returned by any API. A minimum length is enforced on the server.
- **Sessions:** HttpOnly, SameSite session cookies with expiry and CSRF protection. Protected endpoints return 401 or 403 correctly, and tests prove it. In automated tests Google is replaced by a stub, and the real token check is covered by unit tests with sample tokens.
- **Privacy and data protection:** public response schemas exclude reporter identity and the room label; internal staff notes are never returned to students. Only necessary data is collected, a consent notice is shown at signup, and users can delete their account. Complaint records are anonymised, not removed.
- **Input safety:** ORM or parameterised queries only, server-side validation and length limits on every input, and rate limiting on sensitive operations (login, registration, verification, password reset, draft requests).
- **Review:** the application will be checked against the OWASP Top 10, and the results listed in the final report.

7. Technology Stack and Deployment

Layer	Choice
Backend	Python with FastAPI; OpenAPI 3.x specification delivered as a YAML file
Database	MySQL 8 with SQLAlchemy ORM and Alembic migrations
Frontend	HTML, CSS and JavaScript, served by the backend container

Layer	Choice
AI	Gemini API (`gemini-2.5-flash-lite`), called from the backend only
Authentication	Email and password (Argon2 hashes) with email verification and password reset; Google Sign-In (ID token verified on the backend); server-side session cookies
Email	SMTP account set through environment variables; log output in demonstration mode
Testing	pytest and pytest-cov for unit and integration tests; Playwright for end-to-end tests
CI	GitHub Actions with Ruff linting, all tests and coverage on every push and pull request, using a MySQL service container
Packaging	Docker and Docker Compose

The whole stack runs in two containers: a MySQL 8 container and an application container that serves both the FastAPI backend and the frontend files. A single “docker compose up” starts the database, applies the migrations, loads the seed data and starts the application, so that someone with only Docker installed can run the site after cloning the repository.